

The software stack

Every tool this book leans on, positioned in one map — what it is for, when it enters the story, and the version discipline that keeps a run reproducible

Chapter 4 promised this map. The training stack is not one tool but five layers of them, and most confusion about "what should we use" dissolves once each tool is pinned to its layer and its chapter. This appendix is the reference: the layer diagram, one table of what the book uses where, and the version discipline that Chapter 9 made non-negotiable. It is deliberately opinionated — these are the defaults this book's pipelines are built on, not a survey — and each row points back to the chapter where the choice is argued rather than asserted.

A.1 The five layers

THE STACK, POSITIONED (bottom = closest to the silicon; right column = where it is argued)

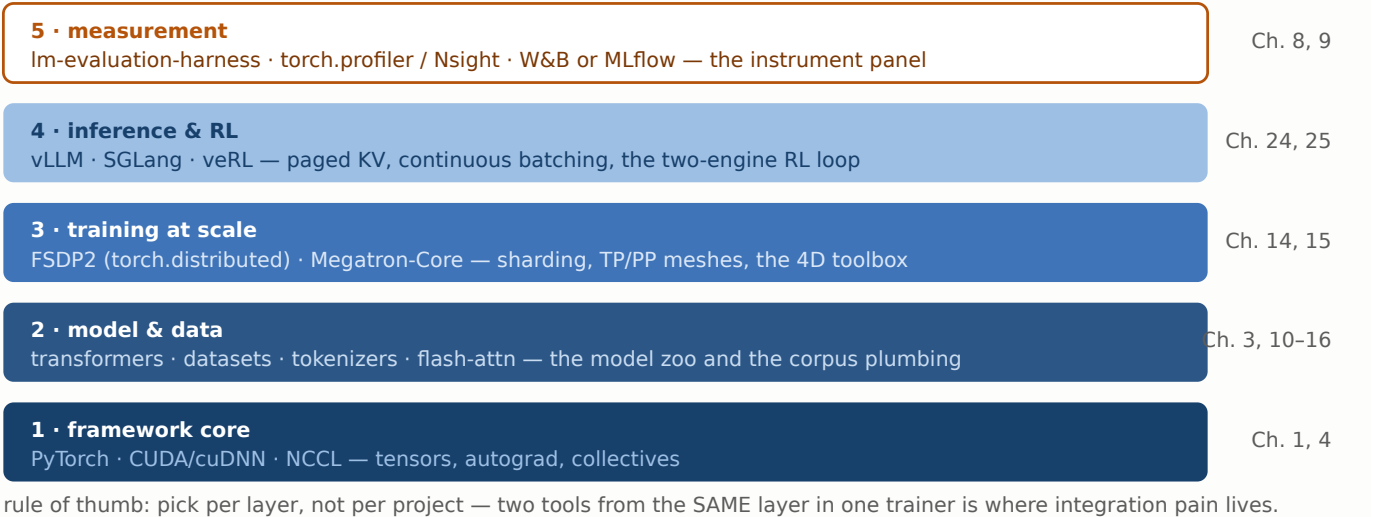


Figure A.1 — The stack in five layers. Lower layers change rarely and break loudly; upper layers change often and break quietly — which is why the version pinning of §A.3 concentrates its paranoia at the top.

A.2 What the book uses where

Table A.1 — The default stack, tool by tool. "Alternative" names the credible substitute and when it wins.

TOOL	JOB IN THIS BOOK	ENTERS AT	ALTERNATIVE, AND WHEN IT WINS
------	------------------	-----------	-------------------------------

TOOL	JOB IN THIS BOOK	ENTERS AT	ALTERNATIVE, AND WHEN IT WINS
PyTorch (≥ 2.4)	everything — tensors, autograd, <code>torch.compile</code> , FSDP2, DTensor/device mesh	Ch. 1	JAX — TPU shops and research on program transforms
transformers	model definitions, checkpoints, chat templates	Ch. 3	hand-rolled model code — when you need Ch. 13-style kernel surgery
datasets + tokenizers	streaming corpus plumbing; BPE training (Ch. 11)	Ch. 2, 10	WebDataset/Mosaic streaming — petabyte-scale shard pipelines
flash-attn	fused attention + varlen kernels that make packing free	Ch. 13, 16	PyTorch SDPA — smaller deps, slightly behind on features
FSDP2	the book's default sharding for $\leq \sim 256$ GPUs	Ch. 14	Megatron-Core — TP/PP at trillion-token scale; DeepSpeed — legacy ZeRO stacks
vLLM	rollout engine: paged KV, continuous batching	Ch. 25	SGLang — structured generation and radix-cache reuse
veRL	GRPO/PPO with the hybrid two-engine loop	Ch. 25	TRL — single-node SFT/DPO simplicity, no hybrid engine
lm-evaluation-harness	the frozen suite of Table 9.1, pinned configs	Ch. 9	OLMES configs on top of it — when comparability with OLMo matters
torch.profiler / Nsight Systems	the MFU audit of §8.6–8.8	Ch. 8	—
W&B / MLflow	curves, configs, artifact lineage	Ch. 4	TensorBoard — air-gapped clusters

A.3 Version discipline: the lockfile is part of the run

Chapter 9 established the rule for evaluation; it holds for the whole stack: *a result travels with its versions or it does not travel*. The failure modes are concrete — a `transformers` minor release changes a chat template and every eval shifts (§9.7's cliff); a flash-attn wheel built for the wrong CUDA silently falls back to a slow path and MFU drops eight points with no error; an lm-eval upgrade rewrites a task template and your GSM8K "regresses." The practice that prevents all three costs one file and one habit:

```
# the environment IS an artifact of the run – freeze it next to the checkpoint
pip freeze > runs/2026-09-21_7b/requirements.lock.txt
python - <<'PY' >> runs/2026-09-21_7b/env.json
import torch, json
print(json.dumps({
    "torch": torch.__version__, "cuda": torch.version.cuda,
    "cudnn": torch.backends.cudnn.version(),
    "gpu": torch.cuda.get_device_name(0),
    "nccl": ".".join(map(str, torch.cuda.nccl.version()))}))
PY
# and the habit (Ch. 9): after ANY upgrade, rerun one frozen old checkpoint
# through the suite before believing numbers from a new one.
```

Two containerization notes from our own runs. Build one image per *project phase*, not per experiment — the pre-training image (layers 1–3) and the post-training image (adds vLLM/veRL, whose CUDA-graph and NCCL expectations are the usual conflict source) want to evolve separately. And upgrade on a schedule, not on release day: the stack's top layers move monthly, and the productive rhythm is a quarterly pinned refresh with the Chapter 9 control-rerun as the gate, plus out-of-band upgrades only for a bug you are actually hitting.

Referenced from Chapter 4 (§4.8). Companion install recipes and images: the course repository, [env/](#) .